

```
import { useState, useRef, useCallback } from "react";

// — Data —————

const WORKOUT_PLANS = {
  gym: {
    label: "Full Gym",
    days: [
      {
        name: "Chest & Triceps",
        tag: "PUSH",
        color: "#FF6B35",
        exercises: [
          { name: "Barbell Bench Press", sets: "4", reps: "6-8", note: "Primary h" },
          { name: "Incline Dumbbell Press", sets: "3", reps: "8-10", note: "Upper h" },
          { name: "Cable Fly", sets: "3", reps: "12-15", note: "Constant tension" },
          { name: "Tricep Rope Pushdown", sets: "3", reps: "10-12", note: "" },
          { name: "Skull Crushers", sets: "3", reps: "10-12", note: "EZ-bar" },
        ],
      },
      {
        name: "Back & Biceps",
        tag: "PULL",
        color: "#4ECDC4",
        exercises: [
          { name: "Deadlift", sets: "4", reps: "5-6", note: "Compound strength" },
          { name: "Weighted Pull-ups", sets: "4", reps: "6-8", note: "Add belt we" },
          { name: "Seated Cable Row", sets: "3", reps: "10-12", note: "" },
          { name: "Dumbbell Curl", sets: "3", reps: "10-12", note: "Supinate at t" },
          { name: "Face Pulls", sets: "3", reps: "15-20", note: "Rear delt health" },
        ],
      },
      {
        name: "Legs & Glutes",
        tag: "LEGS",
        color: "#A855F7",
        exercises: [
          { name: "Barbell Back Squat", sets: "4", reps: "6-8", note: "ATG depth" },
          { name: "Romanian Deadlift", sets: "3", reps: "8-10", note: "Hamstring" },
          { name: "Leg Press", sets: "3", reps: "10-12", note: "" },
          { name: "Walking Lunges", sets: "3", reps: "12 each", note: "" },
          { name: "Calf Raises", sets: "4", reps: "15-20", note: "Full range" },
        ],
      },
    ],
  },
};
```

```

{
  name: "Shoulders & Core",
  tag: "SHOULDERS",
  color: "#F59E0B",
  exercises: [
    { name: "Seated DB Shoulder Press", sets: "4", reps: "8-10", note: "" }
    { name: "Lateral Raises", sets: "4", reps: "12-15", note: "Slow eccentric" },
    { name: "Front Raises", sets: "3", reps: "12", note: "" },
    { name: "Hanging Leg Raises", sets: "3", reps: "15", note: "" },
    { name: "Cable Crunch", sets: "3", reps: "15-20", note: "" },
  ],
},
],
},
bodyweight: {
  label: "No Equipment",
  days: [
    {
      name: "Chest & Triceps",
      tag: "PUSH",
      color: "#FF6B35",
      exercises: [
        { name: "Diamond Push-ups", sets: "4", reps: "12-15", note: "Tricep emphasis" },
        { name: "Wide Push-ups", sets: "3", reps: "15-20", note: "Chest stretch" },
        { name: "Pike Push-ups", sets: "3", reps: "10-12", note: "Shoulder + triceps" },
        { name: "Tricep Dips (chair)", sets: "3", reps: "12-15", note: "" },
        { name: "Archer Push-ups", sets: "3", reps: "8 each", note: "Unilateral" },
      ],
    },
    {
      name: "Back & Biceps",
      tag: "PULL",
      color: "#4ECDC4",
      exercises: [
        { name: "Pull-ups / Chin-ups", sets: "4", reps: "8-12", note: "Use door pull-up bar" },
        { name: "Inverted Rows (table)", sets: "3", reps: "12-15", note: "" },
        { name: "Superman Hold", sets: "3", reps: "15 reps/30s", note: "Lower back" },
        { name: "Towel Bicep Curl", sets: "3", reps: "12", note: "Isometric resistance" },
        { name: "Reverse Snow Angels", sets: "3", reps: "15", note: "Rear delts" },
      ],
    },
    {
      name: "Legs & Glutes",
      tag: "LEGS",
      color: "#A855F7",
      exercises: [
        { name: "Bulgarian Split Squat", sets: "4", reps: "10 each", note: "Reactive" },
      ],
    },
  ],
},

```

```

    { name: "Jump Squats", sets: "3", reps: "15", note: "Explosive power" }
    { name: "Single-Leg RDL", sets: "3", reps: "10 each", note: "Balance +
    { name: "Glute Bridge Holds", sets: "3", reps: "30s", note: "" },
    { name: "Wall Sit", sets: "3", reps: "45-60s", note: "" },
  ],
},
{
  name: "Shoulders & Core",
  tag: "SHOULDERS",
  color: "#F59E0B",
  exercises: [
    { name: "Handstand Hold (wall)", sets: "3", reps: "20-30s", note: "" },
    { name: "Pike Push-ups", sets: "4", reps: "12-15", note: "Overhead pres
    { name: "Plank", sets: "3", reps: "60s", note: "" },
    { name: "Side Plank", sets: "3", reps: "45s each", note: "" },
    { name: "V-ups", sets: "3", reps: "15", note: "" },
  ],
},
],
},
};

```

```
const MACRO_TARGETS = { calories: 2000, protein: 180, carbs: 200, fat: 60 };
```

```
const SAMPLE_FOOD_LOG = [
  { id: 1, name: "Chicken Breast (6oz)", cal: 280, protein: 52, carbs: 0, fat: 6,
  { id: 2, name: "Brown Rice (1 cup)", cal: 215, protein: 5, carbs: 45, fat: 2, t
  { id: 3, name: "Greek Yogurt", cal: 130, protein: 17, carbs: 9, fat: 3, time: "
];

```

```
// ——— Helpers —————
```

```
function MacroRing({ value, max, color, label, unit = "g" }) {
  const pct = Math.min(value / max, 1);
  const r = 28, cx = 34, cy = 34, stroke = 5;
  const circumference = 2 * Math.PI * r;
  const offset = circumference * (1 - pct);
  return (
    <div style={{ display: "flex", flexDirection: "column", alignItems: "center",
    <svg width={68} height={68}>
      <circle cx={cx} cy={cy} r={r} fill="none" stroke="#1e293b" strokeWidth={s
      <circle cx={cx} cy={cy} r={r} fill="none" stroke={color} strokeWidth={str
        strokeDasharray={circumference} strokeDashoffset={offset}
        strokeLinecap="round"
        style={{ transformOrigin: "center", transform: "rotate(-90deg)", transi
      <text x={cx} y={cy - 4} textAnchor="middle" fill="#f1f5f9" fontSize="11"
      <text x={cx} y={cy + 9} textAnchor="middle" fill="#64748b" fontSize="8" f
    
```

```

        </svg>
        <span style={{ fontSize: 10, color: "#64748b", textTransform: "uppercase",
    </div>
    );
}

function ProgressBar({ value, max, color }) {
    return (
        <div style={{ height: 5, background: "#1e293b", borderRadius: 3, overflow: "h
            <div style={{ height: "100%", width: `${Math.min(value / max * 100, 100)}%`
            </div>
        );
    }
}

// — AI Scan Modal —————

function ScanModal({ onClose, onAdd }) {
    const [step, setStep] = useState("idle"); // idle | scanning | result | manual
    const [imagePreview, setImagePreview] = useState(null);
    const [result, setResult] = useState(null);
    const [manualName, setManualName] = useState("");
    const [error, setError] = useState("");
    const fileRef = useRef();

    const analyzeImage = async (base64Data, mediaType) => {
        setStep("scanning");
        setError("");
        try {
            const res = await fetch("https://api.anthropic.com/v1/messages", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({
                    model: "claude-sonnet-4-20250514",
                    max_tokens: 1000,
                    messages: [{
                        role: "user",
                        content: [
                            { type: "image", source: { type: "base64", media_type: mediaType, d
                            { type: "text", text: `Analyze this food image and estimate nutriti
{"name":"food name","serving":"serving size estimate","calories":number,"protein"
All macros in grams. Be realistic with estimates based on visible portion size.`
                        ]
                    }]
                })
            });
            const data = await res.json();
            const text = data.content?.map(b => b.text || "").join("").trim();

```

```

        const parsed = JSON.parse(text.replace(/```json|```/g, "").trim());
        setResult(parsed);
        setStep("result");
    } catch (e) {
        setError("Couldn't analyze image. Try again or enter manually.");
        setStep("idle");
    }
};

const handleFile = (file) => {
    if (!file) return;
    const reader = new FileReader();
    reader.onload = (e) => {
        const dataUrl = e.target.result;
        const base64 = dataUrl.split(",")[1];
        const mediaType = file.type;
        setImagePreview(dataUrl);
        analyzeImage(base64, mediaType);
    };
    reader.readAsDataURL(file);
};

const analyzeText = async () => {
    if (!manualName.trim()) return;
    setStep("scanning");
    setError("");
    try {
        const res = await fetch("https://api.anthropic.com/v1/messages", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({
                model: "claude-sonnet-4-20250514",
                max_tokens: 500,
                messages: [{
                    role: "user",
                    content: `Estimate nutrition for: "${manualName}". Respond ONLY with
{"name":"food name","serving":"standard serving","calories":number,"protein":numb
                }]
            })
        });
        const data = await res.json();
        const text = data.content?.map(b => b.text || "").join("").trim();
        const parsed = JSON.parse(text.replace(/```json|```/g, "").trim());
        setResult(parsed);
        setStep("result");
    } catch (e) {
        setError("Couldn't analyze. Try again.");
    }
};

```

```

        setStep("manual");
    }
};

const confidenceColor = { high: "#4ECDC4", medium: "#F59E0B", low: "#FF6B35" };

return (
    <div style={{ position: "fixed", inset: 0, background: "rgba(0,0,0,0.85)", zI
    <div style={{ background: "#0f172a", border: "1px solid #1e293b", borderRadius
    <button onClick={onClose} style={{ position: "absolute", top: 16, right:
    <div style={{ fontFamily: "'Bebas Neue', sans-serif", fontSize: 26, color
    <div style={{ color: "#64748b", fontSize: 13, marginBottom: 24 }}>Take a

    {step === "idle" && (
        <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
            <div onClick={() => fileRef.current?.click()}
                style={{ border: "2px dashed #334155", borderRadius: 14, padding: 3
                onMouseEnter={e => e.currentTarget.style.borderColor = "#FF6B35"}
                onMouseLeave={e => e.currentTarget.style.borderColor = "#334155"}>
                    <div style={{ fontSize: 36, marginBottom: 8 }}>📷</div>
                    <div style={{ color: "#94a3b8", fontSize: 14 }}>Upload food photo</
                    <div style={{ color: "#475569", fontSize: 12, marginTop: 4 }}>JPG,
                    <input ref={fileRef} type="file" accept="image/*" style={{ display:
                </div>
            <div style={{ display: "flex", alignItems: "center", gap: 10, color:
                <div style={{ flex: 1, height: 1, background: "#1e293b" }} />OR<div
            </div>
            <div style={{ display: "flex", gap: 8 }}>
                <input value={manualName} onChange={e => setManualName(e.target.val
                    onKeyDown={e => e.key === "Enter" && analyzeText()}
                    placeholder="e.g. grilled chicken breast 6oz"
                    style={{ flex: 1, background: "#1e293b", border: "1px solid #3341
                <button onClick={analyzeText} style={{ background: "#FF6B35", borde
            </div>
            {error && <div style={{ color: "#FF6B35", fontSize: 13 }}>{error}</di
        </div>
    )}

    {step === "scanning" && (
        <div style={{ textAlign: "center", padding: "40px 0" }}>
            {imagePreview && <img src={imagePreview} alt="" style={{ width: "100%
            <div style={{ display: "flex", justifyContent: "center", gap: 6, marg
                {[0, 1, 2].map(i => (
                    <div key={i} style={{ width: 8, height: 8, borderRadius: "50%", b
                )}}
            </div>
            <div style={{ color: "#94a3b8", fontSize: 14 }}>Analyzing nutrition..

```

```

    </div>
  )}

  {step === "result" && result && (
    <div>
      {imagePreview && <img src={imagePreview} alt="" style={{ width: "100%
      <div style={{ display: "flex", justifyContent: "space-between", align
        <div>
          <div style={{ color: "#f1f5f9", fontWeight: 700, fontSize: 17 }}>
          <div style={{ color: "#64748b", fontSize: 12, marginTop: 2 }}>{re
        </div>
        <div style={{ background: confidenceColor[result.confidence] + "22"
      </div>
      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr 1fr"
        [[
          { label: "Calories", value: result.calories, unit: "kcal", color:
          { label: "Protein", value: result.protein, unit: "g", color: "#4E
          { label: "Carbs", value: result.carbs, unit: "g", color: "#A855F7
          { label: "Fat", value: result.fat, unit: "g", color: "#F59E0B" },
        ].map(m => (
          <div key={m.label} style={{ background: "#1e293b", borderRadius:
            <div style={{ color: m.color, fontFamily: "'DM Mono', monospace
              <div style={{ color: "#64748b", fontSize: 9, textTransform: "up
                </div>
            )}}
          </div>
          {result.notes && <div style={{ background: "#1e293b", borderRadius: 1
          <div style={{ display: "flex", gap: 10 }}>
            <button onClick={() => { setStep("idle"); setImagePreview(null); se
              style={{ flex: 1, background: "#1e293b", border: "1px solid #3341
            <button onClick={() => { onAdd(result); onClose(); }}
              style={{ flex: 2, background: "#FF6B35", border: "none", borderRa
          </div>
        </div>
      )}
    </div>
    <style>{@keyframes pulse { 0%,100%{opacity:.3;transform:scale(0.8)} 50%{op
  </div>
);
}

```

// — Workout Detail —————

```

function WorkoutDetail({ day, onBack }) {
  const [completed, setCompleted] = useState({});
  const toggle = (i) => setCompleted(p => ({ ...p, [i]: !p[i] }));
  const doneCount = Object.values(completed).filter(Boolean).length;

```

```

return (
  <div style={{ animation: "slideIn 0.3s ease" }}>
    <button onClick={onBack} style={{ background: "none", border: "none", color:
    <div style={{ display: "flex", alignItems: "center", gap: 12, marginBottom:
      <div style={{ background: day.color + "22", border: `1px solid ${day.color
        <div style={{ color: "#f1f5f9", fontFamily: "'Bebas Neue', sans-serif", f
      </div>
    <div style={{ color: "#64748b", fontSize: 13, marginBottom: 24, fontFamily:
    <div style={{ height: 4, background: "#1e293b", borderRadius: 2, marginBott
      <div style={{ height: "100%", width: `${doneCount / day.exercises.length
    </div>
    <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
      {day.exercises.map((ex, i) => (
        <div key={i} onClick={() => toggle(i)}
          style={{ background: completed[i] ? "#0f2a1e" : "#0f172a", border: `1
          <div style={{ width: 28, height: 28, borderRadius: "50%", border: `2px
            {completed[i] && <span style={{ color: "fff", fontSize: 14 }}>✓</span>
          </div>
        <div style={{ flex: 1 }}>
          <div style={{ color: completed[i] ? "#86efac" : "#f1f5f9", fontWeig
            {ex.note && <div style={{ color: "#475569", fontSize: 12, marginTop
          </div>
        <div style={{ textAlign: "right" }}>
          <div style={{ color: day.color, fontFamily: "'DM Mono', monospace",
          </div>
        </div>
      </div>
    </div>
  );
}

```

// — Main App —

```

export default function App() {
  const [tab, setTab] = useState("nutrition");
  const [equipment, setEquipment] = useState("gym");
  const [selectedDay, setSelectedDay] = useState(null);
  const [showScan, setShowScan] = useState(false);
  const [foodLog, setFoodLog] = useState(SAMPLE_FOOD_LOG);
  const [weight, setWeight] = useState("");
  const [weightLog, setWeightLog] = useState([
    { date: "May 1", w: 208 }, { date: "May 5", w: 206 }, { date: "May 10", w: 20
    { date: "May 14", w: 204 }, { date: "May 18", w: 203 },
  ]);

```



```

const totals = foodLog.reduce((acc, f) => ({
  cal: acc.cal + f.cal, protein: acc.protein + f.protein,
  carbs: acc.carbs + f.carbs, fat: acc.fat + f.fat,
}), { cal: 0, protein: 0, carbs: 0, fat: 0 });

const addFood = (item) => {
  setFoodLog(prev => [...prev, {
    id: Date.now(), name: item.name,
    cal: item.calories, protein: item.protein, carbs: item.carbs, fat: item.fat
    time: new Date().toLocaleTimeString([], { hour: "2-digit", minute: "2-digit"
  }]]);
};

const logWeight = () => {
  if (!weight) return;
  const today = new Date().toLocaleDateString("en-US", { month: "short", day: "
  setWeightLog(p => [...p, { date: today, w: parseFloat(weight) }]);
  setWeight("");
};

const plan = WORKOUT_PLANS[equipment];

// sparkline
const wMin = Math.min(...weightLog.map(w => w.w)) - 1;
const wMax = Math.max(...weightLog.map(w => w.w)) + 1;
const svgW = 280, svgH = 70;
const toX = (i) => (i / (weightLog.length - 1)) * svgW;
const toY = (w) => svgH - ((w - wMin) / (wMax - wMin)) * svgH;
const polyline = weightLog.map((p, i) => `${toX(i)},${toY(p.w)}`).join(" ");

const NAV = [
  { id: "nutrition", label: "NUTRITION", icon: "🥗" },
  { id: "workout", label: "WORKOUT", icon: "💪" },
  { id: "progress", label: "PROGRESS", icon: "📈" },
];

return (
  <div style={{ minHeight: "100vh", background: "#060d1a", color: "#f1f5f9", fo
    <style>{`
      @import url('https://fonts.googleapis.com/css2?family=Bebas+Neue&family=D
      * { box-sizing: border-box; margin: 0; padding: 0; }
      ::-webkit-scrollbar { width: 4px; } ::-webkit-scrollbar-track { backgroun
      @keyframes slideIn { from { opacity: 0; transform: translateX(20px); } to
      @keyframes fadeUp { from { opacity: 0; transform: translateY(16px); } to
    `}</style>

    {/* Header */}

```

```

<div style={{ background: "linear-gradient(135deg, #0f172a 0%, #060d1a 100%
  <div style={{ fontFamily: "'Bebas Neue', sans-serif", fontSize: 32, lette
    <div style={{ color: "#475569", fontSize: 12, fontFamily: "'DM Mono', mon
  </div>

  { /* Content */ }

  <div style={{ padding: "20px 16px", maxWidth: 560, margin: "0 auto", animat

    { /* - NUTRITION TAB - */ }
    { tab === "nutrition" && (
      <div>
        { /* Calorie ring + macros */ }
        <div style={{ background: "#0f172a", border: "1px solid #1e293b", bor
          <div style={{ display: "flex", justifyContent: "space-between", ali
            <div>
              <div style={{ color: "#64748b", fontSize: 11, textTransform: "u
                <div style={{ display: "flex", alignItems: "baseline", gap: 6,
                  <span style={{ fontFamily: "'Bebas Neue', sans-serif", fontSi
                    <span style={{ color: "#334155", fontSize: 18 }}> {MACRO_TAR
                  </span>
                </div>
              <div style={{ color: "#22c55e", fontSize: 13, marginTop: 4 }}>{
            </div>
            <button onClick={() => setShowScan(true)}
              style={{ background: "linear-gradient(135deg, #FF6B35, #ef4444)
                 SCAN
            </button>
          </div>
        <div>
          <ProgressBar value={totals.cal} max={MACRO_TARGETS.calories} color=
          <div style={{ display: "flex", justifyContent: "space-around", marg
            <MacroRing value={totals.protein} max={MACRO_TARGETS.protein} col
            <MacroRing value={totals.carbs} max={MACRO_TARGETS.carbs} color="
            <MacroRing value={totals.fat} max={MACRO_TARGETS.fat} color="#F59
          </div>
        </div>
      </div>

      { /* Food Log */ }
      <div style={{ display: "flex", justifyContent: "space-between", align
        <div style={{ fontFamily: "'Bebas Neue', sans-serif", fontSize: 22,
          <button onClick={() => setShowScan(true)}
            style={{ background: "#1e293b", border: "1px solid #334155", bord
        </div>
        <div style={{ display: "flex", flexDirection: "column", gap: 8 }}>
          {foodLog.map((f, i) => (
            <div key={f.id} style={{ background: "#0f172a", border: "1px soli
              <div style={{ flex: 1 }}>
                <div style={{ color: "#e2e8f0", fontWeight: 600, fontSize: 14
                  <div style={{ color: "#475569", fontSize: 12, fontFamily: "'D

```

```

    </div>
    <div style={{ display: "flex", gap: 12, alignItems: "center" }}
      {[{ v: f.protein, c: "#4ECDC4", l: "P" }, { v: f.carbs, c: "#
        <div key={m.l} style={{ textAlign: "center" }}>
          <div style={{ color: m.c, fontFamily: "'DM Mono', monospa
            <div style={{ color: "#475569", fontSize: 9 }}>{m.l}</div
          </div>
        )]]}
      <div style={{ background: "#FF6B3522", borderRadius: 8, paddi
    </div>
  </div>
  )))
</div>

{/* Diet plan info */}
<div style={{ background: "#0a1628", border: "1px solid #1e3a5f", bor
  <div style={{ fontFamily: "'Bebas Neue', sans-serif", fontSize: 18,
    {[
      ["Daily Calories", "2,000 kcal", "~500 cal deficit"],
      ["Protein Target", "180g / day", "1g per lb bodyweight"],
      ["Carbs Target", "200g / day", "Performance fuel"],
      ["Fat Target", "60g / day", "Hormonal health"],
    ]}.map(([label, val, sub]) => (
      <div key={label} style={{ display: "flex", justifyContent: "space
        <div>
          <div style={{ color: "#94a3b8", fontSize: 13 }}>{label}</div>
          <div style={{ color: "#475569", fontSize: 11 }}>{sub}</div>
        </div>
        <div style={{ color: "#f1f5f9", fontFamily: "'DM Mono', monospa
      </div>
    )]]}
  </div>
</div>
))}

{/* - WORKOUT TAB - */}
{tab === "workout" && (
  <div>
    {selectedDay ? (
      <WorkoutDetail day={plan.days[selectedDay]} onBack={() => setSelect
    ) : (
      <>
        {/* Equipment Toggle */}
        <div style={{ background: "#0f172a", border: "1px solid #1e293b",
          {[["gym", "🏋️ Full Gym"], ["bodyweight", "🏠 No Equipment"]].ma
          <button key={k} onClick={() => setEquipment(k)}
            style={{ flex: 1, background: equipment === k ? "#FF6B35" :

```

```

        {label}
      </button>
    )))
  </div>

  <div style={{ fontFamily: "'Bebas Neue', sans-serif", fontSize: 1

  <div style={{ display: "flex", flexDirection: "column", gap: 10 }
    {plan.days.map((day, i) => (
      <div key={i} onClick={() => setSelectedDay(i)}
        style={{ background: "#0f172a", border: `1px solid ${day.co
        onMouseEnter={e => { e.currentTarget.style.borderColor = da
        onMouseLeave={e => { e.currentTarget.style.borderColor = da
      <div style={{ width: 44, height: 44, borderRadius: 12, back
        DAY<br />{i + 1}
      </div>
      <div style={{ flex: 1 }}>
        <div style={{ color: "#f1f5f9", fontWeight: 600, fontSize
        <div style={{ color: "#475569", fontSize: 12, marginTop:
        </div>
        <div style={{ display: "flex", alignItems: "center", gap: 8
          <div style={{ background: day.color + "22", border: `1px
            <span style={{ color: "#334155", fontSize: 18 }}></span>
          </div>
        </div>
      </div>
    )))
  </div>

  <div style={{ background: "#0a1628", border: "1px solid #1e3a5f",
    <div style={{ color: "#4ECDC4", fontFamily: "'Bebas Neue', sans
      [{"Rest 60-90s between sets for max muscle stimulus", "Progress
      <div key={i} style={{ display: "flex", gap: 10, padding: "7px
        <span style={{ color: "#FF6B35", flexShrink: 0 }}></span>{
      </div>
    </div>
  </div>
</>
  </div>
  </div>

  {/* - PROGRESS TAB - */}
  {tab === "progress" && (
    <div>
      {/* Weight tracker */}
      <div style={{ background: "#0f172a", border: "1px solid #1e293b", bor
        <div style={{ fontFamily: "'Bebas Neue', sans-serif", fontSize: 20,

```

```

<div style={{ display: "flex", justifyContent: "space-between", ali
  <div>
    <div style={{ color: "#64748b", fontSize: 11, fontFamily: "'DM
      <div style={{ fontFamily: "'Bebas Neue', sans-serif", fontSize:
        <div style={{ color: "#64748b", fontSize: 13 }}>lbs</div>
      </div>
    <div style={{ textAlign: "right" }}>
      <div style={{ color: "#64748b", fontSize: 11, fontFamily: "'DM
        <div style={{ fontFamily: "'Bebas Neue', sans-serif", fontSize:
          <div style={{ color: "#22c55e", fontSize: 12 }}>of 20 lbs goal<
        </div>
      </div>
    </div>
  {/* Sparkline */}
  <svg width="100%" viewBox={`0 0 ${svgW} ${svgH}`} style={{ display:
    <defs>
      <linearGradient id="lg" x1="0" y1="0" x2="0" y2="1">
        <stop offset="0%" stopColor="#4ECDC4" stopOpacity="0.3" />
        <stop offset="100%" stopColor="#4ECDC4" stopOpacity="0" />
      </linearGradient>
    </defs>
    <polygon points={`0,${svgH} ${polyline} ${svgW},${svgH}`} fill="u
    <polyline points={polyline} fill="none" stroke="#4ECDC4" strokeWi
    {weightLog.map((p, i) => (
      <circle key={i} cx={toX(i)} cy={toY(p.w)} r={4} fill="#4ECDC4"
    )))}
  </svg>
  <div style={{ display: "flex", justifyContent: "space-between", mar
    {weightLog.map((p, i) => (
      <div key={i} style={{ color: "#475569", fontSize: 10, fontFamil
    )))}
  </div>
</div>

{/* Log weight */}
<div style={{ display: "flex", gap: 10, marginBottom: 20 }}>
  <input type="number" value={weight} onChange={e => setWeight(e.targ
    style={{ flex: 1, background: "#0f172a", border: "1px solid #1e29
  <button onClick={logWeight}
    style={{ background: "#4ECDC4", border: "none", borderRadius: 12,
</div>

{/* Goal progress */}
<div style={{ background: "#0f172a", border: "1px solid #1e293b", bor
  <div style={{ fontFamily: "'Bebas Neue', sans-serif", fontSize: 20,
  <div style={{ marginBottom: 20 }}>
    <div style={{ display: "flex", justifyContent: "space-between", m
      <span style={{ color: "#94a3b8", fontSize: 14 }}>Weight Loss (2

```

```

        <span style={{ color: "#22c55e", fontFamily: "'DM Mono', monosp
    </div>
    <ProgressBar value={weightLog[0].w - weightLog[weightLog.length -
    </div>
    {[
      { label: "Weekly Workouts", val: 4, max: 5, color: "#FF6B35" },
      { label: "Protein Goal Hit", val: 5, max: 7, color: "#4ECDC4" },
      { label: "Calorie Deficit Days", val: 6, max: 7, color: "#A855F7"
    ].map(s => (
      <div key={s.label} style={{ marginBottom: 16 }}>
        <div style={{ display: "flex", justifyContent: "space-between",
          <span style={{ color: "#94a3b8", fontSize: 14 }}>{s.label}</s
          <span style={{ color: s.color, fontFamily: "'DM Mono', monosp
        </div>
        <ProgressBar value={s.val} max={s.max} color={s.color} />
      </div>
    )))
  <div style={{ background: "#0a1628", borderRadius: 12, padding: 14,
    <div style={{ color: "#F59E0B", fontFamily: "'DM Mono', monospace
      <div style={{ color: "#94a3b8", fontSize: 13 }}>At current pace (
    </div>
  </div>
</div>
  )}
</div>

  { /* Bottom Nav */}
  <div style={{ position: "fixed", bottom: 0, left: 0, right: 0, background:
    {NAV.map(n => (
      <button key={n.id} onClick={() => { setTab(n.id); setSelectedDay(null);
        style={{ flex: 1, background: "none", border: "none", cursor: "pointe
        <span style={{ fontSize: 22 }}>{n.icon}</span>
        <span style={{ fontFamily: "'DM Mono', monospace", fontSize: 10, lett
          {tab === n.id && <div style={{ width: 20, height: 2, background: "#FF
        </button>
      )})
    </div>

    {showScan && <ScanModal onClose={() => setShowScan(false)} onAdd={addFood}
  </div>
  );
}

```